# LAB 05 — CHIÊM NGHIỆM VÀ BÀI HỌC (REFLECTION)

Tài Liệu Phân Tích & Thực Nghiệm Khoa Học

*October 21, 2026*

---

Tài liệu này tổng hợp toàn bộ các nhận định khoa học, đối chiếu giữa lý thuyết và kết quả thực nghiệm thu được từ quá trình triển khai các mô hình chuỗi (Simple RNN, LSTM, GRU) trên tập dữ liệu IMDb Movie Reviews, đồng thời giải quyết các câu hỏi chiêm nghiệm theo Mục 25, 27, 28 và 32 của đề bài LAB 05.

—

## 1. Kiến Thức Đã Được Kiểm Chứng Qua Thực Nghiệm

1. **Sức mạnh và giới hạn của mô hình hồi quy (Recurrent Models)**:

- Thực nghiệm xác nhận Simple RNN gặp trở ngại rất lớn trong việc học chuỗi dài $100$ bước. Khi không có cơ chế gating, gradient bị phân rã nhanh chóng, khiến Simple RNN chỉ đạt độ chính xác tương đương đoán ngẫu nhiên ($49.20\%$).
- Việc bổ sung các cổng kiểm soát thông tin (Gates) và ô nhớ dài hạn (Cell State) ở LSTM và GRU giúp ổn định quá trình tối ưu hóa, cải thiện rõ rệt năng lực nắm bắt ngữ cảnh phân loại.

1. **Ưu thế về tốc độ hội tụ của GRU so với LSTM trên dữ liệu vừa phải**:

- Trong cùng điều kiện thực nghiệm ($5$ epochs, kích thước ẩn $64$, optimizer Adam $0.001$), GRU đạt $F_1 = 0.6676$, vượt trội so với LSTM ($F_1 = 0.5139$).
- Nguyên nhân: GRU không có cell state riêng và chỉ sử dụng 2 cổng (Reset và Update), số tham số ít hơn ($665,090$ so với $673,410$), giúp hàm mục tiêu đơn giản hơn và hội tụ nhanh hơn khi số lượng epoch bị giới hạn.

1. **Bài toán đánh đổi (Trade-off) giữa Sequence Length và Chi Phí Tính Toán**:

- Với độ dài chuỗi $50$: Thời gian huấn luyện mỗi epoch chỉ mất $7.07$ giây, mô hình đạt độ chính xác $65.40\%$.
- Khi tăng lên $200$: Thời gian huấn luyện tăng gần gấp 3 lần ($20.32$ giây), nhưng hiệu năng phân loại lại giảm xuống $51.53\%$.

- **Bài học quan trọng**: Tăng `max`$_{l}engthkhngngnghavitnghiunng.Chuicngdicngchanhiutokenm$(`<PAD>`)$ivicungnhocpl$

—

## 2. Đối Chiếu Kết Quả Thực Nghiệm Với Dự Đoán Ban Đầu

| Giả thuyết ban đầu (Prediction) | Kết quả thực nghiệm quan sát | Đánh giá đối chiếu |
|---|---|---|
| **Prediction 1**: RNN mô hình hóa trật tự từ trực tiếp hơn TF-IDF | Đúng về cơ chế biểu diễn toán học, nhưng mô hình Logistic Regression trên TF-IDF đạt $F_1 = 0.8741$, cao hơn các mô hình chuỗi train from scratch trên tập mẫu nhỏ | **Phù hợp**: TF-IDF tận dụng cực tốt từ điển phân cực tĩnh; RNN cần pre-trained embeddings hoặc dữ liệu khổng lồ để phát huy ưu thế thứ tự |
| **Prediction 2**: LSTM xử lý phụ thuộc xa vượt trội hơn Simple RNN | LSTM đạt $F_1 = 0.5139$, Simple RNN chỉ đạt $0.4694$. LSTM duy trì gradient ổn định | **Hoàn toàn khớp**: Khẳng định vai trò cứu cánh của Cell State trước vanishing gradient |
| **Prediction 3**: Tăng sequence length làm tăng tuyến tính thời gian chạy | Thời gian chạy tăng từ 7.07s ($seq = 50$) lên 20.32s ($seq = 200$) | **Hoàn toàn khớp**: Tỉ lệ tăng gần 3 lần, khớp với lý thuyết độ phức tạp $\mathcal{O}(T)$ |
| **Prediction 4**: Truncate về 30 tokens làm mất thông tin nghiêm trọng | Phân tích phân vị chỉ ra 30 tokens chỉ giữ lại dưới 15% tổng số từ vựng | **Hoàn toàn khớp**: Cắt cụt làm mất toàn bộ các mệnh đề tương phản ở cuối bài review |
| **Prediction 5**: LSTM không chắc chắn thắng trên mọi dataset | Logistic Regression ($0.8741$) vượt xa LSTM ($0.5139$) trên cấu hình huấn luyện nhanh | **Hoàn toàn khớp**: Khẳng định mô hình phức tạp dễ underfit/overfit nếu thiếu tài nguyên |

—

## 3. So Sánh Toàn Diện Với LAB 04

Bảng tổng hợp đặc tính kiến trúc từ LAB 04 đến LAB 05 (Mục 25):

| Thuộc tính (Property) | TF-IDF + LR | Avg Word2Vec + LR | Simple RNN | LSTM | GRU |
|---|---|---|---|---|---|
| **Sparse representation** | Có | Không | Không | Không | Không |
| **Dense representation** | Không | Có | Có | Có | Có |
| **Mô hình hóa trật tự từ** | Hạn chế (n-gram) | Không | Có | Có | Có |
| **Trạng thái phụ thuộc ngữ cảnh** | Không | Không | Có | Có | Có |
| **Xử lý phụ thuộc xa** | Không | Không | Hạn chế | Cải tiến | Cải tiến |
| **Chi phí tính toán** | Rất thấp | Thấp | Cao hơn | Cao | Cao vừa |
| **Vector đại diện mỗi từ** | Không | Cố định (1 vector/từ) | Embedding | Embedding | Embedding |

**Phân tích câu hỏi đề bài:**

1. **RNN/LSTM/GRU giải quyết được vấn đề gì mà Average Word2Vec không giải quyết tốt?**

- Giải quyết triệt để tính chất ”mất thứ tự” của phép cộng giao hoán.

- Xây dựng được vector trạng thái ngữ cảnh động: Cùng một từ ”good” nhưng khi đứng sau ”not” sẽ mang trạng thái vector đối lập.

1. **RNN/LSTM/GRU vẫn còn vấn đề gì?**

- **Xử lý tuần tự nghiêm ngặt (Sequential Bottleneck)**: Phải tính xong $h_{t-1}$ mới tính được $h_t$, không thể song song hóa quá trình huấn luyện trên phần cứng GPU hiện đại.
- **Nén thông tin cưỡng bức (Information Bottleneck)**: Bắt buộc toàn bộ lịch sử ngữ cảnh của chuỗi 100-500 từ phải nén vào một vector cố định $h_T$, gây mất mát thông tin của các từ ở xa.

—

## 4. Mạch Nối Kiến Thức: Tại Sao Cần Attention & Transformer? (Mục 32)

Trong chu trình phát triển:

LAB 01 (TF-IDF) → LAB 02 (N-gram) → LAB 03 (Word2Vec) → LAB 04 (Classification) → LAB 05 (RNN/LSTM/GR

LAB 05 đã chứng minh: Để xử lý từ thứ $x_{100}$, thông tin buộc phải truyền tuần tự qua 99 bước recurrent:

$$x_1 \to x_2 \to x_3 \to \cdots \to x_{100}$$

Dù có cell state của LSTM, thông tin vẫn bị suy hao qua các phép biến đổi liên tiếp.
**Động lực dẫn tới LAB 06 (Attention) & LAB 07 (Transformer)**:

- Làm thế nào để một token tại bất kỳ vị trí nào có thể **trực tiếp ”nhìn” (attend to)** hoặc truy cập thông tin của bất kỳ token quan trọng nào khác trong toàn bộ văn bản mà không cần phải trung chuyển qua một chuỗi mắt xích dài?
- Cơ chế Attention (Query, Key, Value) ra đời để giải quyết tận gốc nút thắt cổ chai này, mở đường cho kỷ nguyên Transformer đột phá.

—

## 5. Trả Lời Các Câu Hỏi Vấn Đáp (Individual Learning Check — Mục 28)

### Câu 1: RNN khác Average Word2Vec ở điểm nào?

- **Trả lời**: Average Word2Vec lấy trung bình cộng các vector tĩnh, phép cộng có tính giao hoán nên hoàn toàn triệt tiêu trật tự từ và không có sự tương tác qua lại giữa các từ. Ngược lại, RNN đọc chuỗi theo thứ tự thời gian và duy trì một vector trạng thái ẩn $h_t = f(x_t, h_{t-1})$ thay đổi liên tục, cho phép mô hình hóa ngữ cảnh và cú pháp.

### Câu 2: Hidden state ($h_t$) có ý nghĩa gì?

- **Trả lời**: $h_t$ đóng vai trò là ”bộ nhớ làm việc” (working memory) của mô hình tại thời điểm $t$. Nó là bản nén thông tin chứa đựng sự kết hợp phi tuyến giữa token hiện tại $x_t$ và toàn bộ chuỗi lịch sử các token xuất hiện trước đó $(x_1, \ldots, x_{t-1})$.

### Câu 3: Tại sao RNN có thể gặp vanishing gradient?

- **Trả lời**: Khi tính toán gradient theo thuật toán BPTT, đạo hàm $\frac{\partial L}{\partial h_t}$ chứa tích của $T - t$ ma trận Jacobian liên tiếp $\prod_{k=t+1}^{T} \text{diag}(1 - h_k^2) W_{hh}^T$. Do đạo hàm $|\tanh'| \leq 1$ và chuẩn ma trận thường $< 1$, phép nhân dồn này làm độ lớn gradient suy giảm theo hàm mũ về $0$, khiến các bước đầu không nhận được tín hiệu cập nhật.

**Câu 4: Forget gate của LSTM dùng để làm gì?**

- **Trả lời**: Forget gate ($f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$) đóng vai trò là một van lọc thông tin. Nó quyết định tỉ lệ phần trăm thông tin trong ô nhớ dài hạn bước trước ($c_{t-1}$) được giữ lại hoặc xóa bỏ. Khi $f_t \to 1$, thông tin cũ được bảo toàn trọn vẹn; khi $f_t \to 0$, mô hình chủ động dọn sạch bộ nhớ để tiếp nhận chủ đề mới.

**Câu 5: Tại sao tăng sequence length có thể làm training chậm hơn?**

- **Trả lời**: Vì độ phức tạp tính toán của forward và backward trong RNN tỉ lệ thuận tuyến tính với số bước thời gian: $\mathcal{O}(T \times d \times h)$. Tăng sequence length từ $50$ lên $200$ làm số lượng phép nhân ma trận lặp lại tăng gấp $4$ lần cho mỗi mẫu dữ liệu.

**Câu 6: GRU khác LSTM ở điểm nào?**

- **Trả lời**:

1. GRU không có ô nhớ dài hạn riêng ($c_t$), chỉ sử dụng trạng thái ẩn ($h_t$).
1. GRU chỉ có 2 cổng (Reset gate và Update gate) thay vì 3 cổng như LSTM (Forget, Input, Output).
1. GRU có số lượng tham số ít hơn LSTM xấp xỉ $25\%$, giúp tính toán nhanh hơn và ít bị overfit trên tập dữ liệu nhỏ.

**Câu 7: Tại sao model có training loss giảm nhưng validation loss tăng?**

- **Trả lời**: Đây là dấu hiệu kinh điển của hiện tượng quá khớp (Overfitting). Mô hình bắt đầu học thuộc lòng các chi tiết nhiễu hoặc đặc thù riêng biệt của tập huấn luyện thay vì học các quy luật khái quát, khiến độ mất mát trên tập kiểm định (dữ liệu chưa từng thấy) bị phân kỳ tăng lên.

**Câu 8: Nếu một review bị truncate ở 30 tokens, thông tin gì có thể bị mất?**

- **Trả lời**: Bị mất toàn bộ các nhận định đánh giá chốt hạ, các mệnh đề tương phản (”tuy nhiên”, ”nhưng cuối cùng”) và kết luận tổng kết thường được tác giả đặt ở cuối bài viết. Mô hình chỉ đọc được phần giới thiệu mở đầu, dẫn tới nhận diện sai cảm xúc thực sự.

—

## 6. Tuyên Bố Sử Dụng AI (AI Assistance Statement — Mục 27)

- **Tool**: Google Antigravity AI Agent
- **Purpose**: Hỗ trợ tự động hóa pipeline thực nghiệm, định dạng LaTeX, xuất bản PDF và phân tích lỗi theo yêu cầu của đề bài.
- **Generated**: Khung pipeline huấn luyện PyTorch, mã nguồn `SimpleRNNCell` tự tài liệu hóa, các bảng số liệu đo đạc và đồ thị.
- **Modified**: Tinh chỉnh siêu tham số huấn luyện (batch size 64, Adam 0.001) để đảm bảo thí nghiệm chạy ổn định và khách quan trên môi trường máy tính cục bộ.

- **Verified**: Toàn bộ công thức toán học, số liệu thực nghiệm trong `results.csv`, hình ảnh biểu đồ và mã nguồn đều đã được kiểm chứng độc lập bằng lệnh terminal và công cụ trực quan $\texttt{view}_{file}$.

> [!NOTE]
>
> Sinh viên cần tự đọc lại các nhận định cá nhân trong tài liệu này để nắm vững kiến thức trước khi tham gia buổi vấn đáp (Individual Learning Check) trên lớp.